

---

# Re-FORC: Adaptive Reward Prediction for Efficient Chain-of-Thought Reasoning

---

Renos Zabounidis<sup>1,2</sup> Aditya Golatkar<sup>1</sup> Michael Kleinman<sup>1</sup> Alessandro Achille<sup>1</sup> Wei Xia<sup>1</sup> Stefano Soatto<sup>1</sup>

## Abstract

We propose Re-FORC, an adaptive reward prediction method that, given a query, enables prediction of the expected future rewards as a function of the number of future thinking tokens. Re-FORC trains a lightweight adapter on reasoning models, demonstrating improved prediction with longer reasoning and larger models. Re-FORC enables: 1) *early stopping* of unpromising reasoning chains, reducing compute by up to 26% compared to fixed-budget cutoffs, while maintaining accuracy, 2) *optimized model and thinking length selection* that outperforms the largest model alone—reaching 1.7 percentage points higher peak accuracy while needing up to 12% less compute to match the largest model’s accuracy, 3) *adaptive test-time scaling*, which increases accuracy by 9.9 percentage points (on average at maximum compute) over confidence-based baselines. Re-FORC allows dynamic reasoning with length control via cost-per-token thresholds while estimating computation time upfront.

## 1. Introduction

Modern reasoning models can dynamically use inference-time computation to improve answer quality, but determining the optimal amount of inference-time computation for a given query remains an open challenge. While an inference pipeline can generate longer reasoning traces, backtrack to explore alternatives, or even delegate computation to specialized models, the large action space makes it difficult to identify the best strategy for any given query. This challenge is compounded by the diversity of user requirements. Different users have varying tolerance for latency and assign different values to output quality. A strategy that works well

for a time-sensitive application may be entirely inappropriate for a high-stakes decision where accuracy is paramount. How can we adaptively optimize inference-time compute based on both query difficulty and user-specific constraints?

The question can be framed more generally as maximizing the net utility of inference:

$$J = \mathbb{E}[R_{best} - \lambda T_{total}]$$

where  $R_{best}$  represents the reward of the best answer found during inference-time reasoning (i.e., variable-length chain-of-thought computation),  $T_{total}$  is the total compute cost incurred,  $\lambda$  captures the user’s cost sensitivity, and the expectation is over the randomness in inference.

While models can be trained for specific trade-offs, dynamically adapting to user constraints at inference time remains largely unexplored. Crucially, there is no canonical choice of  $\lambda$  when evaluating  $J$ , since the value of an agent depends on the user and the environment (Achille & Soatto, 2026). Consider an agent tasked with determining whether a fruit is edible: if the agent takes so long that the fruit has spoiled, providing the correct answer is of no use. But if the agent is a botanist categorizing species for an encyclopedia, it matters little whether the fruit is edible by the time the answer arrives. In general, an agent must learn to modulate the cost of time depending on the environment and the user’s stated preferences.

Our key insight is that current models lack the **ability to predict the marginal benefit of additional computation** for a given query. Lacking this, we cannot make informed decisions about *when to continue reasoning, when to backtrack, or when the expected improvement no longer justifies the computational cost*.

To address this gap, we introduce Re-FORC, a method for forecasting the reward-versus-compute trade-off both before and during inference in reasoning-based LLMs. Given a partial reasoning trace, Re-FORC predicts the distribution of expected rewards from generating additional thinking tokens. This capability is achieved through a lightweight adapter fine-tuned on top of existing models, enabling predictions for both the base model and external black-box

---

Work done during an internship at AWS Agentic AI. <sup>1</sup>AWS Agentic AI <sup>2</sup>Carnegie Mellon University. Correspondence to: Aditya Golatkar <agolatka@amazon.com>.

*Proceedings of the 43<sup>rd</sup> International Conference on Machine Learning*, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

systems.

Building on Re-FORC, we then introduce a **greedy algorithm for optimal decision-making** inspired by the theory of Pandora’s box problems (Weitzman, 1979). This development is motivated by the following practical application requirements:

- **Adaptive early termination:** By stopping reasoning trajectories when the marginal benefit of additional computation no longer outweighs its cost, Re-FORC reduces computational cost by up to 26% on average compared to fixed-budget cutoffs (S1, Muennighoff et al., 2025) while maintaining accuracy.
- **Joint model and compute optimization:** When multiple model sizes are available, Re-FORC jointly selects both the optimal model and the number of reasoning tokens to maximize net utility for each query, going beyond traditional model routing by explicitly considering compute budgets.
- **User-controlled inference:** Users can specify their cost-performance trade-off  $\lambda$  at inference time without model retraining, providing a more intuitive alternative to token-count specifications while adapting inference to query complexity.
- **Transparent compute estimation:** Re-FORC can provide users with upfront estimates of expected computation time, improving the user experience for latency-sensitive applications.

## 2. Related Work

Our work intersects two key research areas: forecasting methods that predict LLM performance, and efficient reasoning strategies that optimize compute-accuracy trade-offs during inference.

**Forecasting, Probing, and Verifier-Guided Inference-Time Scaling.** Most forecasting research addresses non-chain-of-thought contexts, predicting restart benefits (Manvi et al., 2024) or initial correctness (Damani et al., 2025). Recent work shows models encode future correctness (Zhang et al., 2025; Yoon et al., 2025) and factuality signals (Servadio et al., 2025; Yang & Jia, 2025) during reasoning. While some approaches use external verifiers for intermediate step evaluation (Uscidda et al., 2025; Snell et al., 2025; Wu et al., 2025), our method forecasts future expected reward as a continuous function of additional reasoning tokens, enabling utility-based decisions across reasoning horizons. Concurrent work on adaptive test-time compute allocation shows that compute-optimal scaling policies (Snell et al., 2025) and reward-guided adaptive reasoning depth (Cui et al., 2025) can outperform naive scaling approaches. Tree-of-thought methods enable deliberate multi-branch search over reasoning states (Long, 2023; Besta et al., 2025) and adaptive

budget allocation across multiple trajectories (Liao et al., 2025), though recent work identifies efficiency challenges in naive verifier-guided exploration (Wang et al., 2025). Recent iterative refinement methods repeatedly generate and aggregate reasoning traces, showing near-monotonic accuracy gains but rapidly growing compute cost (Venkatraman et al., 2025). Our forecaster differs in that it predicts a continuous marginal value curve of extra thinking tokens, rather than heuristically expanding and pruning reasoning paths.

**Efficient Reasoning and Model Selection.** A parallel line of work has developed strategies for efficient reasoning under compute constraints. Self-consistency (Wang et al., 2023b) spawned compute-aware variants that halt when votes converge (Li et al., 2024; Liu & Wang, 2025) or use confidence-weighted aggregation (Taubenfeld et al., 2025). Theoretical analyses characterize diminishing marginal returns from additional samples (Komiya et al., 2026). Recent work on adaptive reasoning length control includes token-budget-aware policies (Yuan et al., 2025) and learned stopping mechanisms that predict when further reasoning becomes redundant (Sun et al., 2025), including dynamic early-exit policies that truncate reasoning once additional steps yield diminishing returns (Yang et al., 2025b). Structured exploration increases search breadth under larger budgets (Bi et al., 2025). At the token level, early exit mechanisms enable anytime generation with calibrated stopping (Schuster et al., 2022), while agentic frameworks learn when to plan (Paglieri et al., 2025; Pan & Zhao, 2025) and strategic decomposition approaches like ReAct (Yao et al., 2023), SCALAR (Zabounidis et al., 2025), least-to-most prompting (Zhou et al., 2023), and plan-and-solve strategies (Wang et al., 2023a) allocate deliberate planning compute. Separately, model routing systems choose among different-sized models to optimize accuracy-cost trade-offs (Jitkritum et al., 2026; Guha et al., 2024; Ding et al., 2025; Yue et al., 2025; Guo et al., 2025). In addition to model selection, recent works aim to control the reasoning length either through training-free (Muennighoff et al., 2025) or training-based approaches (Aggarwal & Welleck, 2025; Kleinman et al., 2025). A complementary line of work uses model-internal confidence signals—such as token-level entropy or distributional confidence—to halt or aggregate reasoning traces, with DeepConf (Fu et al., 2026) being a recent representative; we compare against DeepConf as a strong adaptive baseline. Our approach bridges these areas by using forecasted reward curves to make principled decisions about both when to stop reasoning and which model to use, grounded in metareasoning theory (Russell & Welford, 1991; Weitzman, 1979; Aouad et al., 2025). Recent work frames modern LLM inference as bounded-optimal decision-making over cognitive effort (Fu et al., 2026), providing theoretical grounding for treating compute allocation as an economically principled optimization problem. Addi-

tionally, Achille & Soatto (2026) recently framed inference-time search as a Pandora’s box problem and proposed using forecasted rewards for making cost-aware decisions, which we realize in our work. In concurrent work, Manvi et al. (2026) also train a forecaster and apply it for efficient and adaptive reasoning.

### 3. Methodology

Inference-time compute allocation in reasoning models presents a sequential decision problem where an agent must decide at each step whether to generate additional thinking tokens or terminate. Let  $\mathcal{X}$  denote the query space,  $\mathcal{Z}$  the space of partial reasoning traces (possibly multiple trajectories), and  $\mathcal{Y}$  the output space, and  $\Pi = \{\pi_i\}_{i=1}^N$  be a collection of reasoning models. At each time step, the system state is  $(x, z)$  where  $x \in \mathcal{X}$  is the query,  $z \in \mathcal{Z}$  is the collection of partial traces generated so far, and the agent needs to decide which trajectory to continue or terminate the search.

This problem exhibits the structure of a *Markov chain selection* problem (Scully & Terenin, 2025), where the agent must choose which of multiple stochastic processes (reasoning trajectories for LLMs) to advance. Each reasoning continuation corresponds to a transient Markov chain with reward structure, and the agent must select which chain to progress based on expected net utility. However, unlike classical settings where reward distributions are known, reasoning models operate with unknown, state-dependent reward distributions that depend on query complexity and current reasoning progress. Recently, (Achille & Soatto, 2026) showed that universal search (Levin, 1973) can be formulated as a Pandora’s box problem which can be solved with the Gittins policy. We use the framework from (Achille & Soatto, 2026) in Section 4.2 where we propose the Pandora’s box greedy search for reasoning models using our forecaster  $\psi(t \mid x, z, \pi)$ .

The optimal policy for such problems (Weitzman, 1979) takes the form of a Gittins index policy (Scully & Terenin, 2025; Xie et al., 2024), which assigns each possible continuation a reservation value—the minimum expected reward improvement needed to justify its computational cost. In the Gittins index policy (Scully & Terenin, 2025), we compute the Gittins index of each continuation, and compare it against our estimate of current best reward. We terminate search if none of the continuations improve the current best reward, otherwise we choose the trajectory with highest Gittins index.

The central challenge is that computing Gittins indices requires knowledge of the reward distributions for different reasoning continuations, which are unknown and must be learned. We address this by introducing **Re-FORC**, which

learns to predict the forecasting functional  $\psi(t \mid x, z, \pi)$  that estimates expected rewards from generating  $t$  additional thinking tokens from state  $(x, z)$  using model  $\pi$ . Using our forecaster we can approximate the Gittins index to choose the next action in the Markov chain.

We now introduce the necessary preliminaries and define the forecasting functional that enables training our predictor to approximate the Gittins index.

#### 3.1. Sequential Compute Allocation

We formalize the inference-time compute allocation problem in LLMs by defining the decision space, objectives, and constraints. The core challenge is modeling how reasoning models generate thinking tokens and how the quality of their final answers depends on the computational resources allocated.

We use a Markov decision process where the state space is  $\mathcal{S} = \mathcal{X} \times \mathcal{Z}$  and each state  $s = (x, z)$  represents a query with a partial reasoning trace. The action space  $\mathcal{A}$  includes:

- **Continue reasoning:** Generate  $\Delta$  additional thinking tokens
- **Terminate:** Stop reasoning and output final answer  $y$
- **Switch model:** Transfer to a different reasoning model  $\pi'$  when available

Each reasoning continuation from state  $(x, z)$  can be modeled as advancing a transient Markov chain with reward function  $R : \mathcal{X} \times \mathcal{Y} \rightarrow [0, 1]$  and cost function  $c : \mathcal{A} \rightarrow \mathbb{R}_+$ . The agent’s objective is to maximize expected net utility:

$$J = \mathbb{E}[R_{best} - \lambda \cdot T_{total}] \quad (1)$$

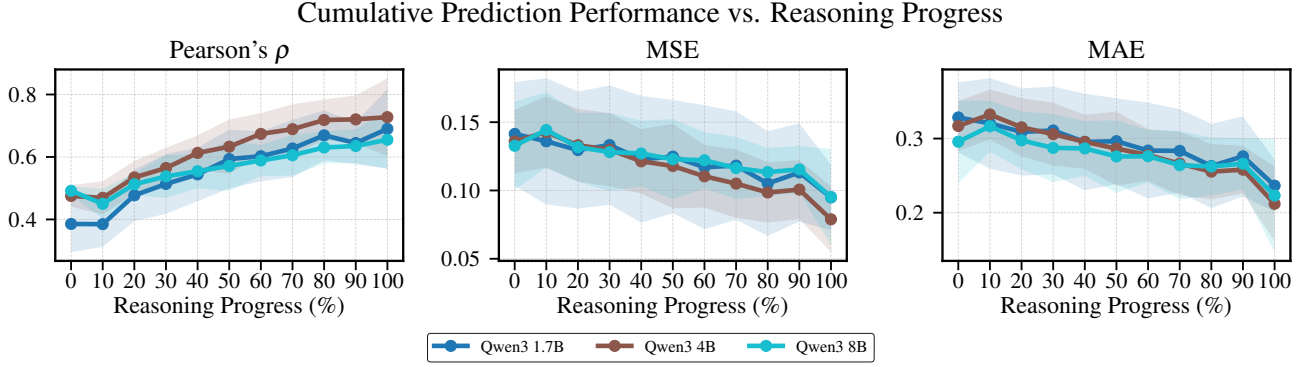
where  $R_{best}$  is the reward of the best answer discovered,  $T_{total}$  is the total computational cost, and  $\lambda > 0$  represents the cost sensitivity parameter. This objective balances exploration of potentially better solutions against computational expenditure measured in (necessarily subjective, environment- and user-dependent) units of  $\lambda$ .

The key challenge distinguishing our setting from classical Markov chain selection problems lies in computing the Gittins index itself. For a Markov chain in state  $s$ , the Gittins index  $g$  for a reasoning trajectory is defined as the solution to:

$$\mathbb{E} \left[ (R(x, y) - g)_+ \mid s = (x, z) \right] - \lambda t = 0 \quad (2)$$

where  $t$  is the number of future reasoning tokens and  $(x)_+ = \max(x, 0)$ . For binary rewards  $R \in \{0, 1\}$  with success probability  $p$ , the Gittins index has closed form:

$$g = 1 - \frac{\lambda t}{p} \quad (3)$$



**Figure 1. Forecast performance with reasoning progress.** Cumulative prediction performance of Re-FORC for Qwen3 (Yang et al., 2025a) models, averaged across four math benchmarks (MATH500, AMC 2024, AIME 2024, and AIME 2025): at each reasoning-progress level, the forecaster is conditioned on the chain-of-thought prefix and predicts the reward (Equation (7)) at all remaining horizons; we report the agreement between these predictions and the empirically realized success rates (Equation (6)) at those horizons—i.e., the quality of the full forecast curve, not just a next-step estimate. **(left)** Pearson correlation  $\rho$  (higher is better); **(middle)** mean squared error (MSE, lower is better); **(right)** mean absolute error (MAE, lower is better). Forecast quality improves with reasoning progress. Per-dataset results are in Appendix B.1.

when  $p > \lambda t$  (positive expected value). Given our forecaster  $\psi(t | x, z, \pi)$  which predicts  $p$  for  $t$ -token continuations, we define the optimal compute budget as:

$$t^*(x, z, \pi) = \arg \min_t \left[ \frac{\lambda t}{\psi(t | x, z, \pi)} \right] \quad (4)$$

The Gittins index for model  $\pi_i$  at state  $(x, z)$  is then:

$$g_i(x, z) = 1 - \frac{\lambda_i t^*(x, z, \pi_i)}{\psi(t^*(x, z, \pi_i) | x, z, \pi_i)} \quad (5)$$

The Gittins index policy selects the option with highest  $g_i$  and terminates when all  $g_i < R_{best}$ , where  $R_{best}$  is the best reward obtained so far.

Generally, computing the Gittins index requires solving Equation (2), which depends on the reward distribution. When the outcome is binary, this reduces to calculating the expected reward  $\mathbb{E}[R(x, y)]$ . This expectation depends on the stochastic reasoning process and the final answer quality, neither of which have closed-form expressions for language models. We address this challenge by learning the forecasting functional  $\psi(t | x, z, \pi) = \mathbb{E}[R(x, y)]$  for  $t$ -token continuations. This functional enables approximate computation of Gittins indices and principled decision-making in our adaptive setting.

### 3.2. Adaptive Reward Prediction

Given a query  $x \in \mathcal{X}$ , a partial chain-of-thought  $z \in \mathcal{Z}$ , and a reasoning model  $\pi$ , we define two modes of inference:  $\pi^{(r)}$  for thinking token generation and  $\pi^{(o)}$  for final output generation. The output given  $t$  additional thinking tokens is obtained by first sampling additional reasoning tokens  $z_t \sim \pi^{(r)}(\cdot | x, z, t)$  where  $|z_t| \leq t$ , then sampling the output  $y \sim \pi^{(o)}(\cdot | x, z, z_t)$ .

Given a reward function  $R(x, y) : \mathcal{X} \times \mathcal{Y} \rightarrow [0, 1]$ , the adaptive forecasting functional is:

$$\psi(t | x, z, \pi) \triangleq \mathbb{E}_{\substack{z_t \sim \pi^{(r)}(\cdot | x, z, t) \\ y \sim \pi^{(o)}(\cdot | x, z, z_t)}} [R(x, y)] \quad (6)$$

This functional represents the expected reward after allocating exactly  $t$  additional thinking tokens, starting from the current reasoning state  $(x, z)$ . Critically,  $\psi$  is query-dependent, path-dependent, and accounts for the stochastic nature of both reasoning generation and final answer sampling.

To predict  $\psi(t | x, z, \pi)$ , we design a lightweight forecasting module that can be attached to existing reasoning models. We model the forecaster output using a Beta( $\alpha, \beta$ ) distribution: it has bounded support on  $[0, 1]$  matching the reward range, captures a wide range of confidence patterns with two parameters, and yields both a mean prediction and a variance estimate for calibrated uncertainty. Numerically, applying `softplus` to the network outputs keeps  $\alpha, \beta > 0$  during training and the log-likelihood is straightforward to optimize.

The forecaster predicts Beta parameters  $[\alpha_\theta(x, z, t), \beta_\theta(x, z, t)]^T$  for each thinking token budget  $t$ . At inference, we use the Beta mean as our point estimate of expected reward:

$$\hat{\psi}(t | x, z, \pi) = \frac{\alpha_\theta(x, z, t)}{\alpha_\theta(x, z, t) + \beta_\theta(x, z, t)} \quad (7)$$

Since  $\psi$  is defined over discrete token budgets  $t \in \mathbb{N}$ , we predict it on a uniform grid  $\mathcal{T} = \{0, \Delta, 2\Delta, \dots, t_{\max}\}$  and obtain values at arbitrary  $t$  by linear interpolation between adjacent grid points. This approach balances computational efficiency with forecasting accuracy.

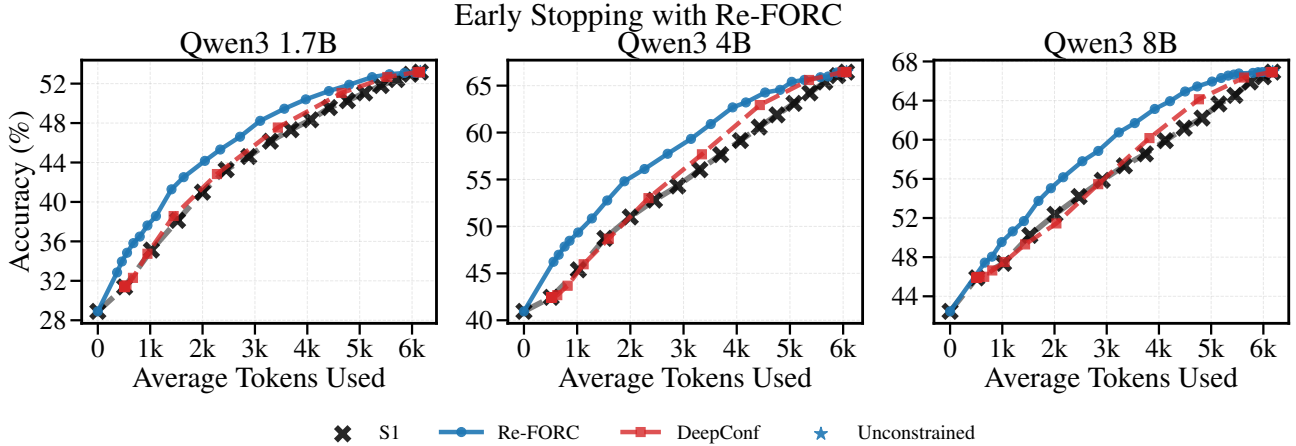


Figure 2. **Early stopping with Re-FORC.** Accuracy vs. average tokens used for Qwen3 1.7B (left), 4B (middle), and 8B (right), averaged across four math benchmarks: MATH500, AMC 2024, AIME 2024, and AIME 2025 (see Section 5.2). Re-FORC (Equation (11)) is compared against S1 (Muennighoff et al., 2025), the recent confidence-based DeepConf baseline (Fu et al., 2026), and unconstrained generation. For the DeepConf baseline, we stop the trajectory early if the confidence of the preceding 512 tokens is below a threshold, and evaluate across thresholds. Re-FORC dominates all baselines at nearly every token budget across all three model sizes, with the gap largest in the moderate-compute regime. The cost-accuracy trade-off is controlled by a single parameter  $\lambda$ . Per-dataset result are in Appendix B.2

### 3.3. Training the predictor

Training the adaptive reward forecaster requires generating a dataset of  $(x, z, t, r)$  tuples where  $r$  represents the true expected reward  $\psi(t | x, z, \pi)$  from continuing reasoning for  $t$  additional tokens from state  $(x, z)$ . We generate training data by sampling problem instances  $(x_i, y_i)$  and generating full unconstrained reasoning trajectories up to the maximum context length. From each complete trajectory, we extract partial traces  $z$  by truncating at regular token intervals corresponding to our forecasting grid  $\mathcal{T} = \{0, \Delta, 2\Delta, \dots, t_{\max}\}$ . For each partial trace  $z$  truncated at position  $\ell$ , we sample the model’s answer directly from state  $(x, z)$ .

When constructing the adaptive forecasting functional  $\psi(t | x, z, \pi)$  for different continuation lengths  $t$ , we reuse these sampled answers by taking all trajectory segments that extend exactly  $t$  tokens beyond the truncation point  $\ell$ . This provides an efficient Monte Carlo approximation through trajectory reuse compared to the alternative of generating multiple continuations for every partial trajectory and every forecasting horizon would require  $O(|\mathcal{T}| \times N \times L)$  trajectory samples, where  $N$  is the number of Monte Carlo samples per estimate and  $L$  is the maximum trajectory length. Our reuse strategy reduces this to  $O(N)$  samples total while maintaining unbiased estimates of  $\psi(t | x, z, \pi)$  across all forecasting horizons.

The forecaster is trained by maximizing the likelihood of observed rewards under the predicted Beta distributions:

$$\mathcal{L}_{\text{forecast}} = \mathbb{E}_{(x, z, t, r) \sim D_{\text{forecast}}} \left[ -\log \text{Beta}(\alpha_{\theta}(x, z, t), \beta_{\theta}(x, z, t))(r) \right] \quad (8)$$

We provide more training details in Section 5. Note that our forecasters are lightweight adapters attached to frozen base reasoning models.

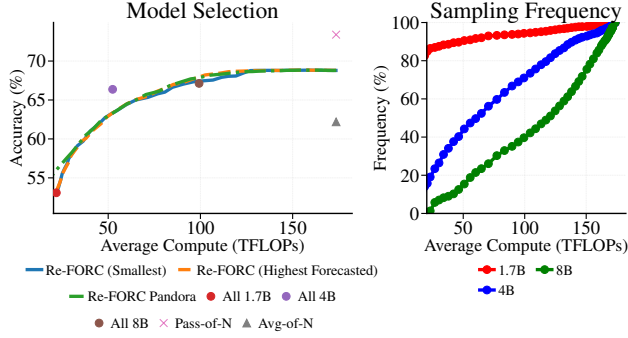
## 4. Compute-Aware Inference

In this section, we explore the applications of our forecaster  $\psi(t | x, z, \pi)$  for optimal inference-time decision-making. We apply the Gittins index-inspired greedy algorithm (Scully & Terenin, 2025; Xie et al., 2024) to evaluate the expected improvement from continuing each available reasoning trace and select the action with highest expected net utility at each step in the search. This approach enables four key applications: (1) early termination decisions that halt partially-completed reasoning traces when marginal improvement no longer justifies compute costs, (2) initial model selection that chooses the optimal model for query processing based on expected cost-accuracy trade-offs, (3) dynamic reselection that transfers unsuccessful queries to models with potentially better capabilities, and (4) exploration-exploitation trade-offs that balance sampling new reasoning traces against continuing existing ones.

### 4.1. Early stopping

Let  $z^*$  be the best (partial) thinking trace, with reward  $R_{\text{best}}$ . Let  $z$  be the current thinking trace from a model  $\pi_i$  (potentially  $z = z^*$ ). If the agent decides to extend  $z$  for additional  $t$  steps, the expected improvement in reward is:

$$\Delta_t J = \mathbb{E}_{\substack{y \sim \pi^{(o)}(\cdot | x, z, z_t) \\ z_t \sim \pi^{(r)}(\cdot | x, z, t)}} [(R(x, y) - R_{\text{best}})_+] - \lambda t \quad (9)$$



**Figure 3. Model and thinking-length selection with Re-FORC.** (Left) Accuracy vs. average compute per problem (TFLOPs) for Qwen3 models averaged across four math benchmarks (MATH500, AMC 2024, AIME 2024, AIME 2025), using the model-selection strategies in Equation (13). The three Re-FORC variants are: *Re-FORC (Smallest)*, which orders models from cheapest to most expensive; *Re-FORC (Highest Forecasted)*, which orders models by decreasing Gittins index; and *Re-FORC Pandora*, which interleaves reasoning across models at the step level (Equation (12)). All Re-FORC variants outperform individual-model baselines in peak accuracy, reaching 1.7 percentage points above All-8B while needing 10% less compute on average to match its accuracy. Pass-of-N is an oracle upper bound. (Right) Sampling frequency of Re-FORC Pandora across model sizes as a function of compute: at minimum compute it preferentially samples the 1.7B model, while at maximum compute it draws from all three. Per-dataset breakdowns in Appendix B.3.

where  $(\cdot)^+$  denotes the positive part—even if the new reward is lower than  $R_{best}$ , we can discard that attempt, but we still pay the compute cost  $\lambda t$ .

For a binary reward distribution, the expected improvement simplifies to:

$$\Delta_t J = \psi(t | x, z, \pi)(1 - R_{best}) - \lambda t \quad (10)$$

We continue reasoning if  $\Delta_t J > 0$  for some  $t^*$ , which is equivalent to continuing iff the Gittins index from Equation (5) exceeds the current best reward:

$$g_i(x, z) = 1 - \frac{\lambda t^*}{\psi(t^* | x, z, \pi)} > R_{best} \quad (11)$$

Otherwise, we stop reasoning. When  $\lambda$  is high or  $R_{best}$  is close to 1, the condition is harder to satisfy, leading to earlier termination. In practice we use our forecaster (Equation (7)) to estimate  $\psi$ .

In Figure 2 we show that early stopping with Re-FORC significantly improves the reward compute trade-off across different sizes of reasoning models averaged across 4 math datasets (Section 5.2). For instance, compared to the S1 baseline (Muennighoff et al., 2025), we save 26% compute on average for the 4B reasoning model while preserving accuracy.

## 4.2. Pandora’s Box Greedy Search

We extend early stopping to the multi-model setting using the Gittins index policy (Achille & Soatto, 2026). Given a collection  $\{\pi_1, \dots, \pi_k\}$  of reasoning models with per-token costs  $\lambda_i$ , and partial traces  $\{z_1, \dots, z_n\}$ , we compute the Gittins index  $g_i(x, z_j)$  for each model-trace pair using Equation (5). At each step, we select the pair with highest Gittins index:

$$(i^*, j^*) = \arg \max_{i, j} g_i(x, z_j) \quad (12)$$

and continue trace  $z_{j^*}$  using model  $\pi_{i^*}$  for  $\Delta$  tokens (1 timestep). We add the new trace to the set of explored trajectories and update  $R_{best}$  if needed. We terminate when all  $g_i(x, z_j) < R_{best}$ , where  $R_{best}$  is the best reward so far. In practice, we use  $\hat{\psi}(t | x, z, \pi)$  to approximate the forecasting functional. With a single model and trace, this reduces to Re-FORC-stopping (Equation (11)).

## 4.3. Model Selection

Model selection is a special case of the Pandora’s box greedy search (Equation (12)) restricted to at most one trajectory per model. The Gittins index ranks models by expected net utility before reasoning begins:

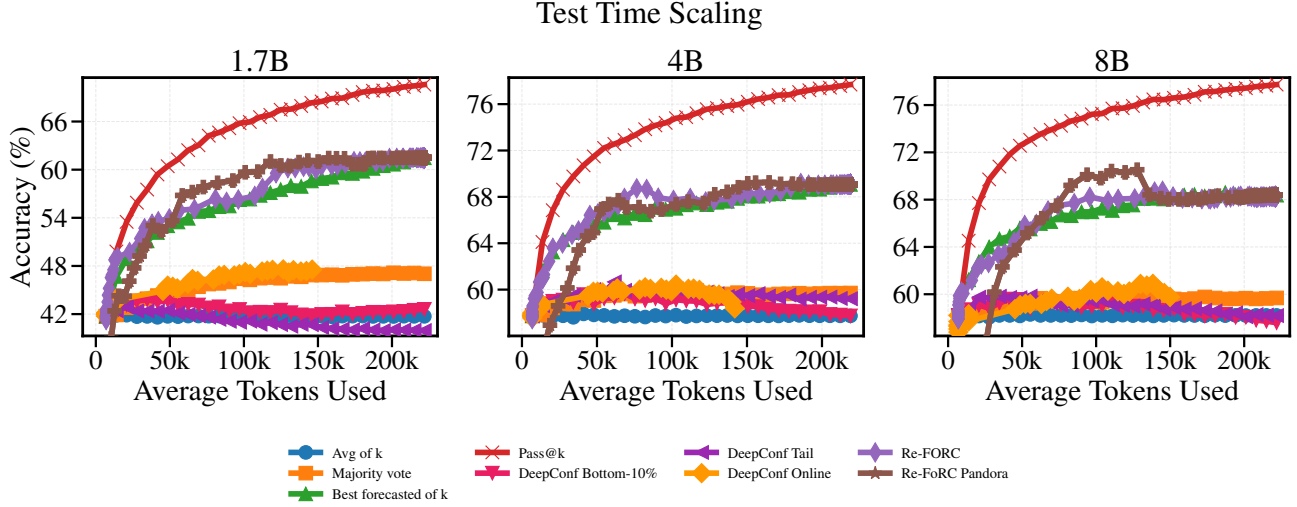
$$j = \arg \max_i g_i(x, \emptyset) \quad (13)$$

The first two approaches choose a model and sample it to completion, differing only in the order each model is considered. *Re-FORC (Highest Forecasted)* orders models by decreasing Gittins index; *Re-FORC (Smallest)* orders by model size, starting with the cheapest model  $\pi_{small}$  and considering the most expensive last. After obtaining reward  $R_{best}$  from one model, both approaches try the next model only if  $g_i(x, \emptyset) > R_{best}$ . The third approach, *Re-FORC Pandora*, applies Equation (12) directly, interleaving reasoning across models at the step level rather than committing to one model at a time.

## 4.4. Test-Time Scaling

Finally, we apply the Pandora’s box greedy search framework to test-time scaling, where we allocate compute across multiple reasoning samples for a given model  $\pi_i$ . Each reasoning trace corresponds to a “box” in the Pandora’s box problem (Section 4.2): opening a box means advancing a trace, and the agent must decide which trace to continue or whether to start a new one. We consider two strategies that differ in the granularity at which boxes are opened.

*Re-FORC-scaling.* Each complete trace is treated as a single open box. Every trace is sampled to completion—no early stopping is applied within a trace—and the decision rule



**Figure 4. Test-time scaling with Re-FORC.** Accuracy vs. average tokens used for Qwen3 1.7B (left), 4B (middle), and 8B (right), averaged across three competition-math benchmarks (AMC 2024, AIME 2024, AIME 2025); we sample  $32 \times$  per query. Per-dataset breakdowns in Appendix B.5. Re-FORC-scaling (Equation (14)) selects a model upfront via Equation (13) and samples traces to completion, deciding only whether to draw another sample, while Re-FORC Pandora (Equation (15)) operates at the step level, dynamically selecting which trace to advance and stopping traces early when they are no longer promising. Both methods are compared against repeated-sampling baselines (Avg-of-k, majority vote, best-forecasted-of-k), three DeepConf (Fu et al., 2026) variants (Online, Tail Conf, Bottom-10% Conf), and the oracle upper bound Pass@k. Best-forecasted-of-k uses the forecaster to select among  $k$  completed samples. The DeepConf variants score traces using the mean of the lowest-decile (10%) of 512-token group confidences within a trace; *Online* gates generation in real time using this score, while *Bottom-10% Conf* filters completed traces and *Tail Conf* uses the mean confidence of the last 2,048 tokens instead. For the offline variants, we select using the highest score, and for the online we select using majority vote from the remaining traces. The online variant additionally terminates sampling before using all  $k$  traces once one answer holds  $\geq 80\%$  of the accepted votes, using the first  $\lceil 0.3k \rceil$  traces as warmup to set the confidence threshold (50th percentile of the warmup scores). At peak compute, Re-FORC outperforms the strongest DeepConf variant (at any compute) by +13.9 pp (+29.3% relative) on 1.7B, +8.4 pp (+13.9%) on 4B, and +7.5 pp (+12.3%) on 8B. Re-FORC Pandora can outperform Re-FORC-scaling in the medium-to-high compute regime ( $\geq 50k$  tokens), while the simpler Re-FORC-scaling is preferable in the very low-compute regime ( $\leq 25k$  tokens).

governs only whether to draw an additional sample: given a query  $x$  and the completed reasoning traces so far, we sample a fresh trace if the Gittins index for a new trace exceeds the best reward among the completed traces,

$$g_i(x, \emptyset) > R_{best} \quad (14)$$

This approach generates complete traces sequentially, deciding after each completed trace whether to resample.

**Re-FORC Pandora scaling.** Rather than treating each trace as an atomic box, we apply the Pandora’s box greedy search at the *step level*: each partial trace  $z_j$  is a box that can be opened incrementally. Given a set of partial traces  $\{z_1, \dots, z_n\}$  for model  $\pi_i$ , at each step we select the trace with highest Gittins index and advance it by  $\Delta$  tokens:

$$j^* = \arg \max_j g_i(x, z_j) \quad \text{s.t.} \quad g_i(x, z_{j^*}) > R_{best} \quad (15)$$

This is a direct application of Equation (12) where each model-trace pair is a box that we open one step at a time. The agent advances trace  $z_{j^*}$  using model  $\pi_i$  for  $\Delta$  tokens, recomputes Gittins indices, and terminates when no continuation improves over the current best reward  $R_{best}$ . Unlike Re-FORC-scaling, which always runs traces to completion,

Pandora applies the early-stopping rule at the step level: a partial trace whose Gittins index falls below  $R_{best}$  is simply never advanced again. By interleaving exploration across partial trajectories at each step, Pandora scaling allocates compute where it is most promising rather than committing to full traces before deciding.

In Figure 4, we show results for all three Qwen-3 model sizes (1.7B, 4B, 8B) averaged across AIME 24/25 and AMC 2024. Both Re-FORC-scaling (Equation (14)) and Re-FORC Pandora (Equation (15)) outperforms the majority-vote baseline across most compute budgets. At peak compute, Re-FORC improves accuracy over the strongest DeepConf variant (at its best-performing compute budget) by 13.9 percentage points for the 1.7B model, 8.4 percentage points for the 4B, and 7.5 percentage points for the 8B. Notably, in the very low compute regime ( $\leq 25k$  tokens), Pandora’s step-level exploration incurs overhead from maintaining multiple partial traces, making it less efficient than the simpler Re-FORC-scaling—the benefits of multi-trajectory interleaving require sufficient compute budget to materialize.

## 5. Experiment details

### 5.1. Training Setup

We implement our adaptive reward forecaster as a lightweight adapter attached to pretrained reasoning models from the Qwen3 family (1.7B, 4B, 8B parameters) (Yang et al., 2025a). The base reasoning models remain frozen during forecaster training to preserve their reasoning capabilities while learning to predict future performance. The forecaster architecture consists of a self-attention pooling layer that takes penultimate-layer activations  $h_{1:n} \in \mathbb{R}^{n \times d}$  and aggregates sequence information into a fixed-size representation, followed by a linear projection head  $g_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^{2|\mathcal{T}|}$  that outputs Beta distribution parameters  $(\alpha_t, \beta_t)$  for each time horizon  $t \in \mathcal{T}$ .

We use a uniform forecasting grid  $\mathcal{T} = \{0, 512, 1024, \dots, 8192\}$  with linear interpolation for intermediate values, where Beta parameters are obtained via softplus activation to ensure positivity. The forecaster introduces minimal computational overhead, requiring only a single forward pass through the base model during training to extract final layer hidden states without generating additional thinking tokens during inference.

Training data is generated by sampling problem instances  $(x_i, y_i)$  from DeepScaleR-Preview (Luo et al., 2025) and creating full unconstrained reasoning trajectories up to a maximum thinking length of 8192 tokens. We extract partial traces at regular intervals and use Monte Carlo estimation with  $N = 8$  samples to compute empirical success rates ( $N = 4$  for the 1.7B model on AIME 2025), with rewards clipped to  $(\varepsilon, 1 - \varepsilon)$  where  $\varepsilon = 10^{-6}$  for numerical stability.

### 5.2. Evaluation Setup

We evaluate on four mathematics reasoning datasets: AMC 2024 (Art of Problem Solving, 2024c;d), Math500 (Lightman et al., 2024), and AIME 2024/25 (Art of Problem Solving, 2024a;b; 2025a;b). Forecasting performance is measured using Pearson correlation ( $\rho$ ), mean squared error (MSE), and mean absolute error (MAE) between predicted and true reward values, while compute-aware inference is evaluated on accuracy-compute trade-offs measuring both final accuracy and total thinking tokens consumed. We use a maximum number of thinking tokens of 8192. Token costs are measured by the number of 512-token reasoning chunks it consumes. We use 32 samples per problem for evaluation (except for Math500, where we use 4 samples).<sup>1</sup>

Our experimental comparisons include unconstrained generation as a standard reasoning baseline without early stopping, fixed token limits representing simple cutoffs with-

out adaptive decision-making, single-model baselines using only the largest or smallest available model, and oracle routing with ground-truth access (Pass@k) as a theoretical upper bound. These baselines allow us to isolate the contributions of adaptive forecasting versus simpler heuristic approaches.

**Baselines.** For each application, we compare Re-FORC against the baselines most appropriate to that setting. **Early stopping** is compared against S1 (Muennighoff et al., 2025), which uses fixed-length token cutoffs, and DeepConf (Fu et al., 2026), a recent confidence-based adaptive reasoning method,<sup>2</sup> alongside unconstrained generation as a no-stopping reference. **Test-time scaling** is additionally compared against majority voting (Wang et al., 2023b), average-of- $k$  sampling, best-forecasted-of- $k$  (which selects the trace with the highest forecasted reward without adaptive stopping), three DeepConf variants (Online, Tail Conf, Bottom-10% Conf), and Pass@ $k$  as the oracle upper bound with ground-truth access. **Model selection** is compared against running each individual model in isolation (All 1.7B, All 4B, All 8B), Avg-of- $N$ , and Pass-of- $N$ . Together, these baselines isolate the contribution of explicit reward forecasting from simpler heuristic and confidence-based approaches.

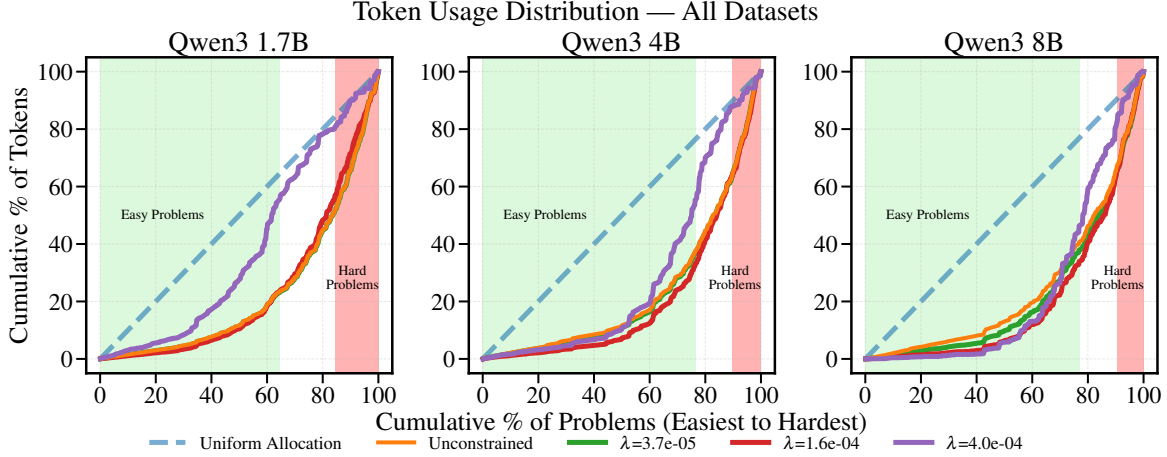
## 6. Results

*User-controlled inference:* Users can dynamically control computational expenditure by selecting a  $\lambda$  value at inference time based on their accuracy and cost requirements; Re-FORC then automatically optimizes the reward-compute trade-off (Figures 2 to 4). The method is most advantageous in intermediate compute regimes, achieving higher accuracy than baselines at equal compute. In early stopping (Figure 2; per-dataset breakdown in Appendix B.2), the maximum accuracy advantage of Re-FORC over S1 at equal compute is +3.8 pp at 1.5k tokens for the 1.7B model, +4.2 pp at 2k tokens for the 4B, and +3.9 pp at 4k tokens for the 8B—larger models benefit from Re-FORC most at higher token budgets. Similarly, in test-time scaling (Figure 4), Re-FORC with the 8B model provides maximum improvements in the 100k-token range.

*Compute-aware applications:* Our experiments demonstrate the effectiveness of Re-FORC across three applications. First, Re-FORC-stopping (Eq. 11) provides smooth accuracy-compute frontiers, where moderate  $\lambda$  values preserve most peak accuracy while reducing reasoning tokens. Re-FORC-stopping reduces compute by 26% on average compared to S1 and 21% compared to DeepConf for the Qwen3 4B model, while maintaining accuracy. Second, Re-FORC-Pandora (Eq. 13) outperforms the largest model alone, reaching 1.7pp higher peak accuracy while needing 12% less compute to match its accuracy. Finally, Re-FORC

<sup>1</sup>For Figure 4, each point at subset size  $k$  averages over up to 100 randomly sampled size- $k$  subsets per problem.

<sup>2</sup>See Appendix B.6 for DeepConf implementation details.



**Figure 5. Token distribution and problem difficulty for Qwen3 models combined across datasets.** Problems are combined across datasets (MATH500, AMC 2024, AIME 2024, AIME 2025) and ordered by per-model solve rate; “easy” problems are solved in  $\geq 90\%$  of trials and “hard” ones in  $< 50\%$ . Each curve shows the cumulative percentage of total tokens spent versus the cumulative percentage of problems, with the dashed diagonal indicating uniform allocation. Increasing  $\lambda$  represents a higher cost sensitivity, encouraging more selective compute use. At high  $\lambda=4.0 \times 10^{-4}$ , the models allocate a smaller proportion of compute to the more difficult problems compared to smaller values of  $\lambda$  and unconstrained generation. For the 8B model, on the easiest  $\sim 40\%$  of problems, increasing  $\lambda$  leads to a smaller percentage of compute used on such problems. Per-dataset results are in Appendix B.4.

Pandora applied to Test Time Scaling (Eq. 14) achieves superior accuracy-compute trade-offs compared to best forecasted in the medium or high compute regime.

*Flexible base model training:* Re-FORC is fully independent of the base reasoning model’s training procedure, in contrast to methods such as L1 (Aggarwal & Welleck, 2025) or e1 (Kleinman et al., 2025) that modify model weights. Operating solely at inference time, it traces the accuracy-compute trade-off curve with fine-grained cost control without altering the base model’s training or architecture.

*Difficulty-based allocation.* We further analyze how Re-FORC-stopping (Eq. 11 and Figure 2) improves token efficiency. We order problems by solve rate and plot the cumulative percentage of tokens used versus the cumulative percentage of problems (Figure 5). At high  $\lambda=4.0 \times 10^{-4}$ , the token allocation curves differ compared to unconstrained generation. Across model sizes, there is a smaller proportion of tokens allocated to the most difficult problems. For the 8B model, the proportion of compute is also reduced on the easiest problems, whereas for the 1.7B model, the proportion increases.

*Forecast accuracy improves with reasoning progress:* Forecast quality improves as chain-of-thought tokens increase, with higher  $\rho$  and lower MSE/MAE (see Figure 1).

## 7. Conclusion

We introduced Re-FORC, an adaptive reward prediction approach that enables efficient control of compute (both model size and reasoning length) over chain-of-thought reasoning by thresholding the forecasting functional using the Gittins

index policy. We formulate the reward-compute prediction problem using Pandora’s box greedy search (Weitzman, 1979; Scully & Terenin, 2025) and provide empirical techniques to approximate the Gittins index policy for reasoning models (Achille & Soatto, 2026) in practice. Our method trains lightweight forecasters (adapters) on top of frozen reasoning models to predict future reward-token trade-offs, enabling three key inference-time applications: (1) early stopping of unpromising reasoning trajectories, (2) compute-aware model selection from a pool of reasoning models, and (3) cost-aware test-time scaling. Results across four math benchmarks demonstrate that forecaster-guided strategies consistently outperform baseline approaches, achieving superior accuracy-compute trade-offs.

**Limitations.** Collecting forecaster training data is computationally expensive, requiring multiple reasoning trajectories per query at various length intervals, with costs scaling with both dataset size and model capacity. Additionally, the forecaster occasionally exhibits overconfidence, predicting higher rewards than the base model can realistically achieve, leading to wasteful continued sampling rather than early termination. Extended training with larger datasets helps mitigate this, though complete calibration remains an ongoing challenge.

## Impact Statement

This work aims to improve the computational efficiency of reasoning in large language models, potentially reducing energy consumption and making advanced AI capabilities more accessible. By enabling adaptive compute allocation, Re-FORC allows systems to avoid wasteful computation

on queries that can be solved with less effort, while appropriately investing resources in more challenging problems. This could democratize access to high-quality AI reasoning by reducing costs.

However, more efficient reasoning systems could also accelerate automation in knowledge work, with potential labor market implications. Additionally, while our method improves efficiency, it does not address the underlying correctness or safety of the reasoning models themselves—a miscalibrated forecaster could prematurely terminate reasoning on important queries or over-allocate compute to adversarial inputs.

## References

- Achille, A. and Soatto, S. Ai agents as universal task solvers. *Entropy*, 28(3), 2026. ISSN 1099-4300. doi: 10.3390/e28030332. URL <https://www.mdpi.com/1099-4300/28/3/332>.
- Aggarwal, P. and Welleck, S. L1: Controlling how long a reasoning model thinks with reinforcement learning. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=4jdIxXBNve>.
- Aouad, A., Ji, J., and Shaposhnik, Y. The pandora’s box problem with sequential inspections, 2025. URL <https://arxiv.org/abs/2507.07508>.
- Art of Problem Solving. 2024 aime i — problems and solutions. [https://artofproblemsolving.com/wiki/index.php/2024\\_AIME\\_I](https://artofproblemsolving.com/wiki/index.php/2024_AIME_I), 2024a. Held January 31–February 1, 2024.
- Art of Problem Solving. 2024 aime ii — problems and solutions. [https://artofproblemsolving.com/wiki/index.php/2024\\_AIME\\_II](https://artofproblemsolving.com/wiki/index.php/2024_AIME_II), 2024b. Held February 7, 2024.
- Art of Problem Solving. 2024 amc 12a — problems and solutions. [https://artofproblemsolving.com/wiki/index.php/2024\\_AMC\\_12A](https://artofproblemsolving.com/wiki/index.php/2024_AMC_12A), 2024c. Administered November 6, 2024.
- Art of Problem Solving. 2024 amc 12b — problems and solutions. [https://artofproblemsolving.com/wiki/index.php/2024\\_AMC\\_12B](https://artofproblemsolving.com/wiki/index.php/2024_AMC_12B), 2024d. Administered November 12, 2024.
- Art of Problem Solving. 2025 aime i — problems and solutions. [https://artofproblemsolving.com/wiki/index.php/2025\\_AIME\\_I](https://artofproblemsolving.com/wiki/index.php/2025_AIME_I), 2025a. Held February 6, 2025.
- Art of Problem Solving. 2025 aime ii — problems and solutions. [https://artofproblemsolving.com/wiki/index.php/2025\\_AIME\\_II](https://artofproblemsolving.com/wiki/index.php/2025_AIME_II), 2025b. Held February 12, 2025.
- Besta, M., Memedi, F., Zhang, Z., Gerstenberger, R., Piao, G., Blach, N., Nyczyk, P., Copik, M., Kwaśniewski, G., Müller, J., Gianinazzi, L., Kubicek, A., Niewiadomski, H., O’Mahony, A., Mutlu, O., and Hoefler, T. Demystifying chains, trees, and graphs of thoughts. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, pp. 1–20, 2025. ISSN 1939-3539. doi: 10.1109/tpami.2025.3598182. URL <http://dx.doi.org/10.1109/TPAMI.2025.3598182>.
- Bi, Z., Han, K., Liu, C., Tang, Y., and Wang, Y. Forest-of-thought: Scaling test-time compute for enhancing LLM reasoning. In Singh, A., Fazel, M., Hsu, D., Lacoste-Julien, S., Berkenkamp, F., Maharaj, T., Wagstaff, K., and Zhu, J. (eds.), *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 4253–4267. PMLR, 13–19 Jul 2025. URL <https://proceedings.mlr.press/v267/bi25a.html>.
- Cui, Y., Dai, Z., He, P., He, B., Liu, H., Tang, X., Zeng, J., Wang, S., Xing, Y., Tang, J., and Dumoulin, B. Adaptive test-time reasoning via reward-guided dual-phase search, 2025. URL <https://arxiv.org/abs/2509.25420>.
- Damani, M., Shenfeld, I., Peng, A., Bobu, A., and Andreas, J. Learning how hard to think: Input-adaptive allocation of LM computation. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=6qUUgw9bAZ>.
- Ding, D., Mallick, A., Zhang, S., Wang, C., Madrigal, D., Garcia, M. D. C. H., Xia, M., Lakshmanan, L. V. S., Wu, Q., and Rühle, V. BEST-route: Adaptive LLM routing with test-time optimal compute. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=tFBIbCVXkG>.
- Fu, Y., Wang, X., Zhang, H., Tian, Y., and Zhao, J. Deep think with confidence. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=8LqHs0KIM7>.
- Guha, N., Chen, M. F., Chow, T., Khare, I. S., and Ré, C. Smoothie: Label free language model routing. In Globerson, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 127645–127672. Curran Associates, Inc., 2024. doi: 10.52202/079017-4053.

- URL [https://proceedings.neurips.cc/paper\\_files/paper/2024/file/e6b57a990462df5afa58d64ce2709db9-Paper-Conference.pdf](https://proceedings.neurips.cc/paper_files/paper/2024/file/e6b57a990462df5afa58d64ce2709db9-Paper-Conference.pdf).
- Guo, D., Yang, D., Zhang, H., Song, J., Wang, P., Zhu, Q., Xu, R., Zhang, R., Ma, S., Bi, X., et al. Deepseek-rl incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, 2025.
- Jitkrittum, W., Narasimhan, H., Rawat, A. S., Juneja, J., Wang, C., Wang, Z., Go, A., Lee, C.-Y., Shenoy, P., Panigrahy, R., Menon, A. K., and Kumar, S. Universal model routing for efficient LLM inference. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=ka82fvJ5f1>.
- Kleinman, M., Trager, M., Achille, A., Xia, W., and Soatto, S. e1: Learning adaptive control of reasoning effort. In *NeurIPS 2025 Workshop on Efficient Reasoning*, 2025. URL <https://openreview.net/forum?id=eGJmUfVgwe>.
- Komiyama, J., Oba, D., and Oyamada, M. Best-of-infinity: Asymptotic performance of test-time LLM ensembling. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=3qiCnLf3jf>.
- Levin, L. A. Universal sequential search problems. *Problems of information transmission*, 9(3):265–266, 1973.
- Li, Y., Yuan, P., Feng, S., Pan, B., Wang, X., Sun, B., Wang, H., and Li, K. Escape sky-high cost: Early-stopping self-consistency for multi-step reasoning. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=ndR8Ytrzh>.
- Liao, B., Dong, H., Xu, Y., Sahoo, D., Monz, C., Li, J., and Xiong, C. Fractured chain-of-thought reasoning, 2025. URL <https://arxiv.org/abs/2505.12992>.
- Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., and Cobbe, K. Let’s verify step by step. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=v8L0pN6EOi>.
- Liu, X. and Wang, L. Answer convergence as a signal for early stopping in reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 17907–17918, 2025.
- Long, J. Large language model guided tree-of-thought, 2023. URL <https://arxiv.org/abs/2305.08291>.
- Luo, M., Tan, S., Wong, J., Shi, X., Tang, W. Y., Roongta, M., Cai, C., Luo, J., Li, L. E., Popa, R. A., and Stoica, I. Deepscaler: Surpassing o1-preview with a 1.5b model by scaling rl. <https://pretty-radio-b75.notion.site/DeepScaleR-Surpassing-O1-Preview-with-a-1-5B-Model-by-Scaling-RL-19681902c1468005bed8ca303013a4e2>, 2025. Notion Blog.
- Manvi, R., Singh, A., and Ermon, S. Adaptive inference-time compute: LLMs can predict if they can do better, even mid-generation, 2024. URL <https://arxiv.org/abs/2410.02725>.
- Manvi, R., Hong, J., Seyde, T., Labonne, M., Lechner, M., and Levine, S. Zero-overhead introspection for adaptive test-time compute. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=GqZYGOYuF2>.
- Muennighoff, N., Yang, Z., Shi, W., Li, X. L., Fei-Fei, L., Hajishirzi, H., Zettlemoyer, L., Liang, P., Candès, E., and Hashimoto, T. B. s1: Simple test-time scaling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 20286–20332, 2025.
- Paglieri, D., Cupiał, B., Cook, J., Piterbarg, U., Tuyls, J., Grefenstette, E., Foerster, J. N., Parker-Holder, J., and Rocktäschel, T. Learning when to plan: Efficiently allocating test-time compute for llm agents, 2025. URL <https://arxiv.org/abs/2509.03581>.
- Pan, B. and Zhao, L. Can past experience accelerate llm reasoning?, 2025. URL <https://arxiv.org/abs/2505.20643>.
- Russell, S. and Wefald, E. Principles of metareasoning. *Artificial intelligence*, 49(1-3):361–395, 1991.
- Schuster, T., Fisch, A., Gupta, J., Dehghani, M., Bahri, D., Tran, V., Tay, Y., and Metzler, D. Confident adaptive language modeling. *Advances in Neural Information Processing Systems*, 35:17456–17472, 2022.
- Scully, Z. and Terenin, A. The gittins index: A design principle for decision making under uncertainty. In *Tutorials in Operations Research: Advances in Analytics and Operations Research: Improving Decisions to Secure the Future*, pp. 28–70. INFORMS, 2025.
- Servedio, G., De Bellis, A., Di Palma, D., Anelli, V. W., and Di Noia, T. Are the hidden states hiding something? testing the limits of factuality-encoding capabilities in llms. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 6089–6104, 2025.

- Snell, C. V., Lee, J., Xu, K., and Kumar, A. Scaling LLM test-time compute optimally can be more effective than scaling parameters for reasoning. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=4FWAwZtd2n>.
- Sun, R., Cheng, W., Li, D., Chen, H., and Wang, W. Stop when enough: Adaptive early-stopping for chain-of-thought reasoning, 2025. URL <https://arxiv.org/abs/2510.10103>.
- Taubenfeld, A., Sheffer, T., Ofek, E., Feder, A., Goldstein, A., Gekhman, Z., and Yona, G. Confidence improves self-consistency in llms. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 20090–20111. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.findings-acl.1030. URL <http://dx.doi.org/10.18653/v1/2025.findings-acl.1030>.
- Uscidda, T., Trager, M., Kleinman, M., Chattopadhyay, A., Xia, W., and Soatto, S. Latts: Locally adaptive test-time scaling. *arXiv preprint arXiv:2509.20368*, 2025.
- Venkatraman, S., Jain, V., Mittal, S., Shah, V., Obando-Ceron, J., Bengio, Y., Bartoldson, B. R., Kailkhura, B., Lajoie, G., Berseth, G., Malkin, N., and Jain, M. Recursive self-aggregation unlocks deep thinking in large language models, 2025. URL <https://arxiv.org/abs/2509.26626>.
- Wang, A., Song, L., Tian, Y., Yu, D., Mi, H., Duan, X., Tu, Z., Su, J., and Yu, D. Don’t get lost in the trees: Streamlining llm reasoning by overcoming tree search exploration pitfalls. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 23946–23959, 2025.
- Wang, L., Xu, W., Lan, Y., Hu, Z., Lan, Y., Lee, R. K.-W., and Lim, E.-P. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*, pp. 2609–2634, 2023a.
- Wang, X., Wei, J., Schuurmans, D., Le, Q. V., Chi, E. H., Narang, S., Chowdhery, A., and Zhou, D. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*, 2023b. URL <https://openreview.net/forum?id=1PL1NIMMrw>.
- Weitzman, M. Optimal search for the best alternative. *Econometrica*, 47(3):641–54, 1979. URL <https://EconPapers.repec.org/RePEc:ecm:emetrp:v:47:y:1979:i:3:p:641-54>.
- Wu, Y., Sun, Z., Li, S., Welleck, S., and Yang, Y. Inference scaling laws: An empirical analysis of compute-optimal inference for LLM problem-solving. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=VNckp7JEHn>.
- Xie, Q., Astudillo, R., Frazier, P., Scully, Z., and Terenin, A. Cost-aware bayesian optimization via the pandora’s box gittins index. *Advances in Neural Information Processing Systems*, 37:115523–115562, 2024.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025a.
- Yang, C., Si, Q., Duan, Y., Zhu, Z., Zhu, C., Li, Q., Chen, M., Lin, Z., and Wang, W. Dynamic early exit in reasoning models, 2025b. URL <https://arxiv.org/abs/2504.15895>.
- Yang, Y. and Jia, R. When do llms admit their mistakes? understanding the role of model belief in retraction, 2025. URL <https://arxiv.org/abs/2505.16170>.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., and Cao, Y. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL [https://openreview.net/forum?id=WE\\_vluYUL-X](https://openreview.net/forum?id=WE_vluYUL-X).
- Yoon, D., Kim, S., Yang, S., Kim, S., Kim, S., Kim, Y., Choi, E., Kim, Y., and Seo, M. Reasoning models better express their confidence. In Belgrave, D., Zhang, C., Lin, H., Pascanu, R., Koniusz, P., Ghassemi, M., and Chen, N. (eds.), *Advances in Neural Information Processing Systems*, volume 38, pp. 103869–103896. Curran Associates, Inc., 2025. URL [https://proceedings.neurips.cc/paper\\_files/paper/2025/file/95ee7181fe256439dca74ec794b33bb6-Paper-Conference.pdf](https://proceedings.neurips.cc/paper_files/paper/2025/file/95ee7181fe256439dca74ec794b33bb6-Paper-Conference.pdf).
- Yuan, D., Wu, D., and Liu, X. Reasoning at the right length: Adaptive budget forcing for efficient and accurate LLM inference, 2025. URL <https://openreview.net/forum?id=ieBgxTG7Mt>.
- Yue, Y., Zhang, G., Liu, B., Wan, G., Wang, K., Cheng, D., and Qi, Y. Masrouter: Learning to route llms for multi-agent systems. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 15549–15572, 2025.
- Zabounidis, R., Wu, Y., Stepputtis, S., Mitchell, T., Li, Y., and Sycara, K. P. SCALAR: Self-supervised composition and learning of skills with LLM planning and

RL. In *NeurIPS 2025 Workshop on Bridging Language, Agent, and World Models for Reasoning and Planning*, 2025. URL <https://openreview.net/forum?id=eDcGexzTQW>.

Zhang, A., Chen, Y., Pan, J., Zhao, C., Panda, A., Li, J., and He, H. Reasoning models know when they’re right: Probing hidden states for self-verification. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=O6I0Av7683>.

Zhou, D., Schärli, N., Hou, L., Wei, J., Scales, N., Wang, X., Schuurmans, D., Cui, C., Bousquet, O., Le, Q. V., and Chi, E. H. Least-to-most prompting enables complex reasoning in large language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=WZH7099tgfM>.

## Appendix

### A. Theoretical Derivations

In this section, we provide derivations for the key equations used in our compute-aware inference framework.

#### A.1. Gittins Index Closed Form

We derive the closed-form Gittins index (Equation (3)) from its definition (Equation (2)) for binary rewards.

**Proposition A.1.** *For binary rewards  $R \in \{0, 1\}$  with success probability  $p = P(R = 1)$ , the Gittins index satisfying  $\mathbb{E}[(R - g)_+] = \lambda t$  has closed form  $g = 1 - \frac{\lambda t}{p}$  when  $p > \lambda t$ .*

*Proof.* For binary  $R \in \{0, 1\}$ :

$$\begin{aligned}\mathbb{E}[(R - g)_+] &= P(R = 1) \cdot (1 - g)_+ + P(R = 0) \cdot (0 - g)_+ \\ &= p \cdot \max(1 - g, 0) + (1 - p) \cdot \max(-g, 0) \\ &= p(1 - g) \quad \text{for } g \in [0, 1]\end{aligned}\tag{16}$$

Setting this equal to  $\lambda t$  per the Gittins definition:

$$\begin{aligned}p(1 - g) &= \lambda t \\ 1 - g &= \frac{\lambda t}{p} \\ g &= 1 - \frac{\lambda t}{p}\end{aligned}\tag{17}$$

This is valid when  $p > \lambda t$ , ensuring  $g > 0$  (positive reservation value).  $\square$

#### A.2. Optimal Horizon Maximizes Gittins Index

We show that the optimal compute budget (Equation (4)) maximizes the Gittins index over possible horizons.

**Proposition A.2.** *The optimal horizon  $t^* = \arg \min_t \frac{\lambda t}{\psi(t)}$  equivalently maximizes the Gittins index  $g(t) = 1 - \frac{\lambda t}{\psi(t)}$ .*

*Proof.* Since  $g(t) = 1 - \frac{\lambda t}{\psi(t)}$ , we have:

$$\begin{aligned}\arg \max_t g(t) &= \arg \max_t \left(1 - \frac{\lambda t}{\psi(t)}\right) \\ &= \arg \min_t \frac{\lambda t}{\psi(t)} = t^*\end{aligned}\tag{18}$$

Thus selecting  $t^*$  that minimizes the cost-to-success ratio equivalently maximizes the reservation value.  $\square$

#### A.3. Expected Improvement for Binary Rewards

We show how the expected improvement (Equation (9)) simplifies for binary rewards.

**Proposition A.3.** *For binary rewards  $R \in \{0, 1\}$  with  $P(R = 1) = \psi(t \mid x, z, \pi)$  and current best reward  $R_{best}$ , the expected improvement is:*

$$\Delta_t J = \psi(t \mid x, z, \pi)(1 - R_{best}) - \lambda t\tag{19}$$

*Proof.* Starting from the general expected improvement:

$$\begin{aligned}\mathbb{E}[(R - R_{best})_+] &= P(R = 1) \cdot (1 - R_{best})_+ + P(R = 0) \cdot (0 - R_{best})_+ \\ &= \psi(t \mid x, z, \pi) \cdot (1 - R_{best}) + 0 \\ &= \psi(t \mid x, z, \pi)(1 - R_{best})\end{aligned}\tag{20}$$

where we use  $(1 - R_{best})_+ = 1 - R_{best}$  since  $R_{best} \in [0, 1]$ , and  $(0 - R_{best})_+ = 0$  since  $R_{best} \geq 0$ . Subtracting the cost term yields the result.  $\square$

#### A.4. Equivalence of Stopping Conditions

We show that the expected improvement condition  $\Delta_t J > 0$  is equivalent to the Gittins threshold rule  $g > R_{best}$ .

**Proposition A.4.** *For binary rewards, continuing reasoning is beneficial ( $\Delta_t J > 0$ ) if and only if the Gittins index exceeds the current best reward ( $g > R_{best}$ ).*

*Proof.* Starting from the condition for positive expected improvement:

$$\begin{aligned}
 \Delta_t J > 0 &\iff \psi(t)(1 - R_{best}) - \lambda t > 0 \\
 &\iff \psi(t)(1 - R_{best}) > \lambda t \\
 &\iff 1 - R_{best} > \frac{\lambda t}{\psi(t)} \\
 &\iff 1 - \frac{\lambda t}{\psi(t)} > R_{best} \\
 &\iff g > R_{best}
 \end{aligned} \tag{21}$$

where we use the closed-form Gittins index  $g = 1 - \frac{\lambda t}{\psi(t)}$  from the previous derivation.  $\square$

## B. Additional Experiments

### B.1. Forecasting performance per dataset

We show the per-dataset forecasting performance corresponding to Figure 1 in the main paper. Each panel reports Pearson correlation  $\rho$ , MSE, and MAE between predicted and true reward as the chain-of-thought progresses, separately for AIME 2024, AIME 2025, AMC 2024, and MATH500.

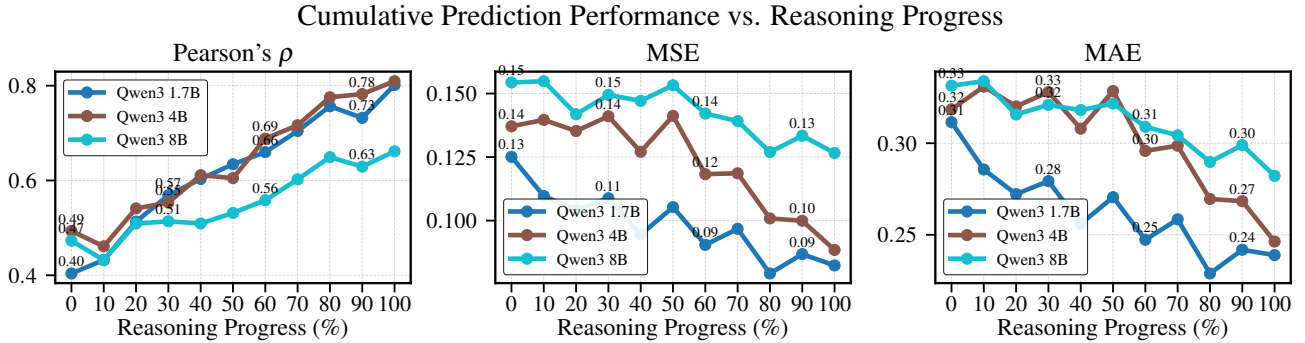


Figure 6. Forecasting performance on AIME 2024.

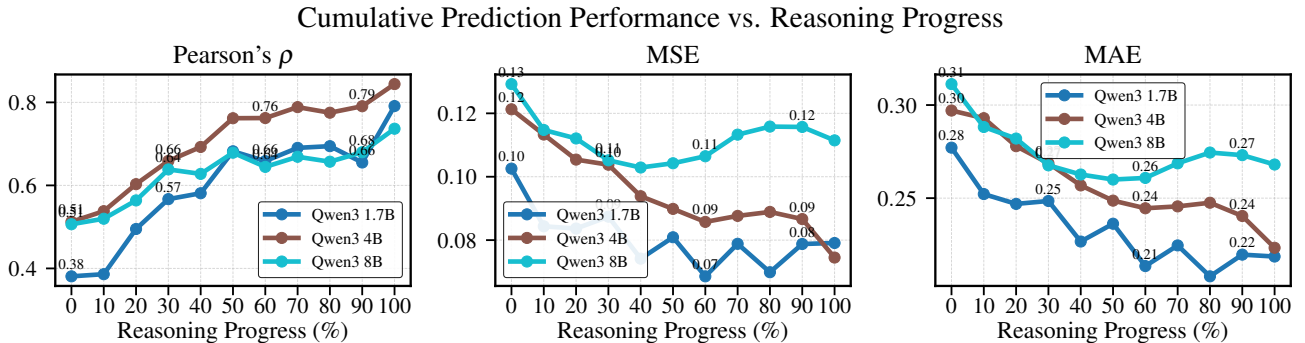


Figure 7. Forecasting performance on AIME 2025.

Cumulative Prediction Performance vs. Reasoning Progress

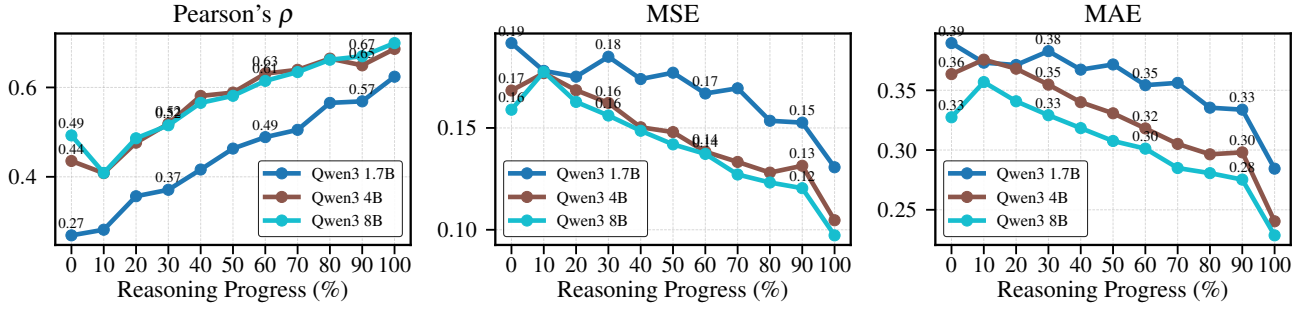


Figure 8. Forecasting performance on AMC 2024.

Cumulative Prediction Performance vs. Reasoning Progress

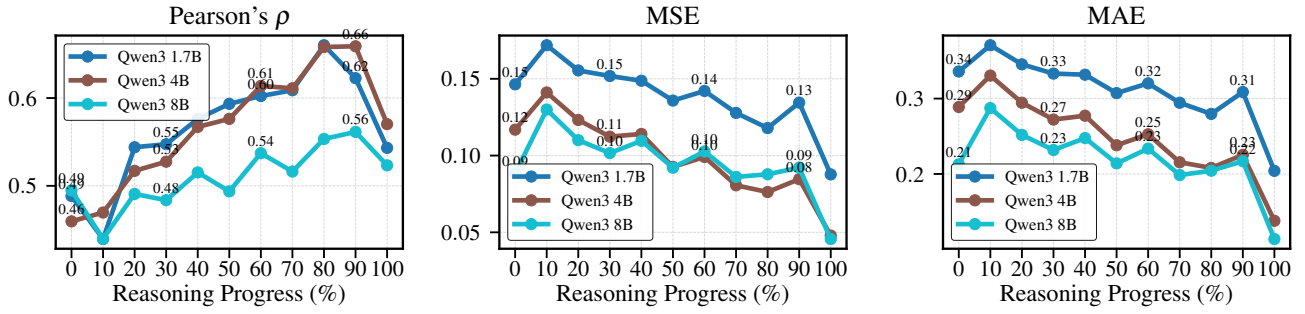


Figure 9. Forecasting performance on MATH500.

## B.2. Early stopping per dataset

Per-dataset breakdown of the early-stopping experiment in Figure 2. Each figure shows accuracy vs. tokens for Qwen3 1.7B, 4B, and 8B side by side, comparing Re-FORC against S1, DeepConf, and unconstrained generation.

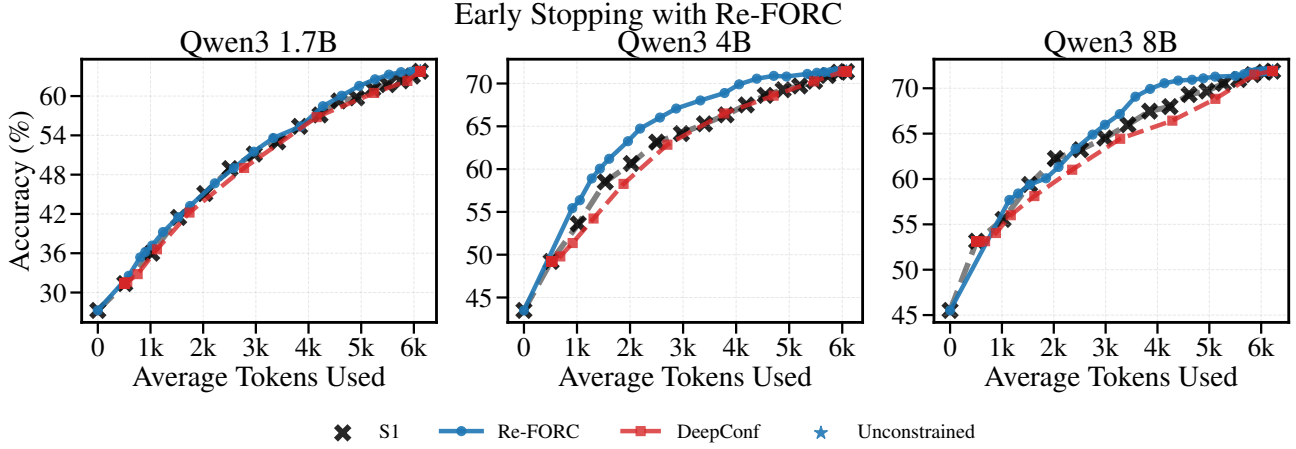


Figure 10. Early stopping on AMC 2024.

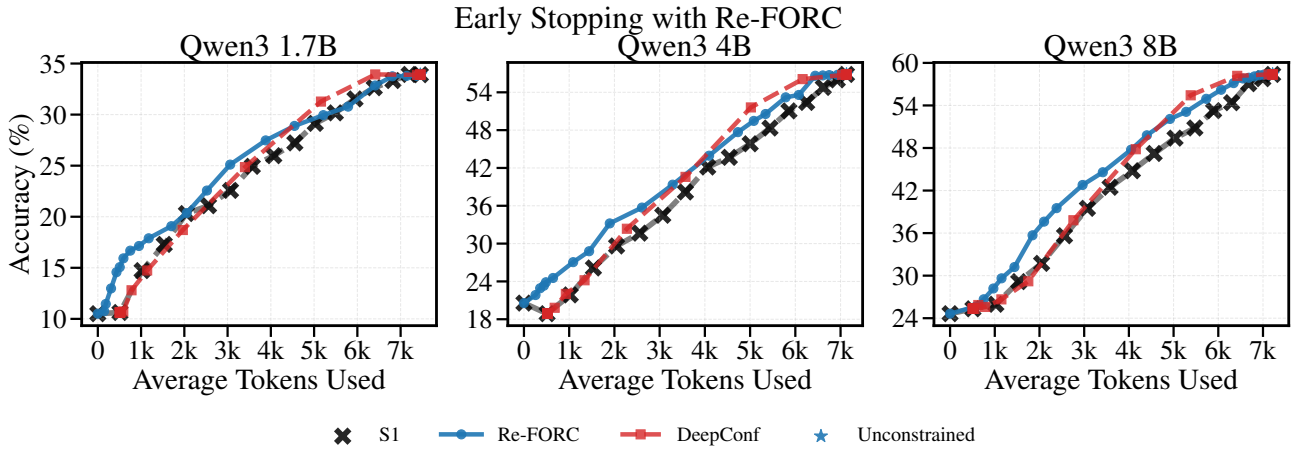


Figure 11. Early stopping on AIME 2024.

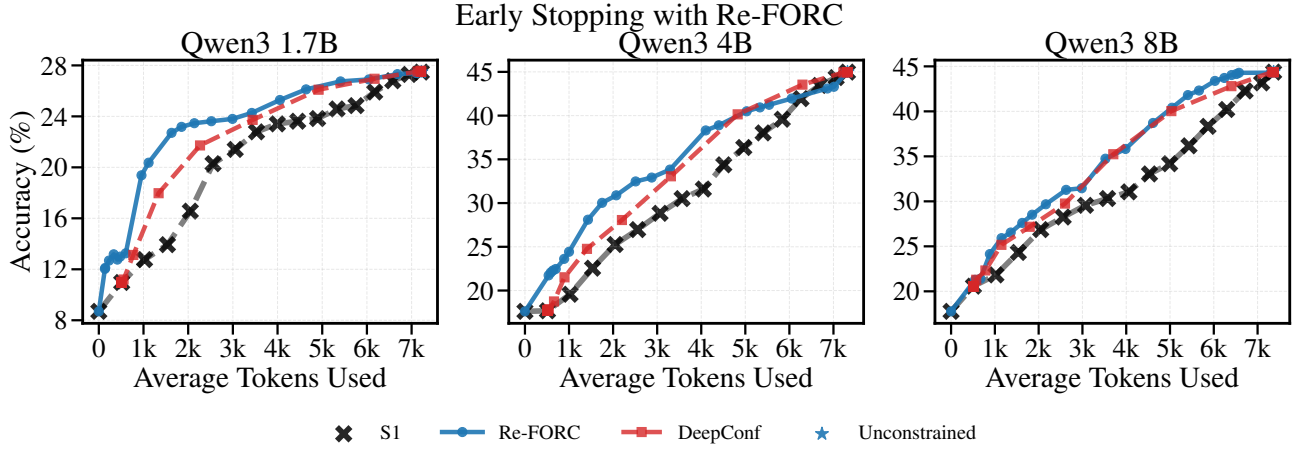


Figure 12. Early stopping on AIME 2025.

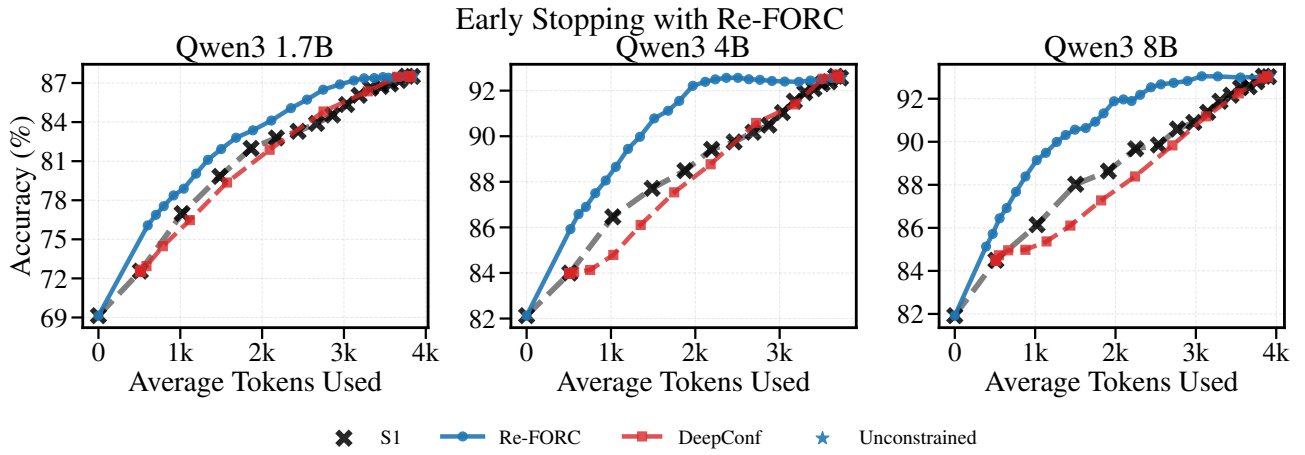


Figure 13. Early stopping on MATH500.

### B.3. Model selection per dataset

Per-dataset breakdown of the model-selection experiment from Figure 3. Each figure shows accuracy vs. average compute (timesteps  $\times$  T-FLOPs) on a single benchmark, comparing the three Re-FORC variants (Smallest-First, Highest-Forecasted-First, Pandora) against the individual-model anchors (All 1.7B / 4B / 8B), Pass-of-N (oracle), and Avg-of-N.

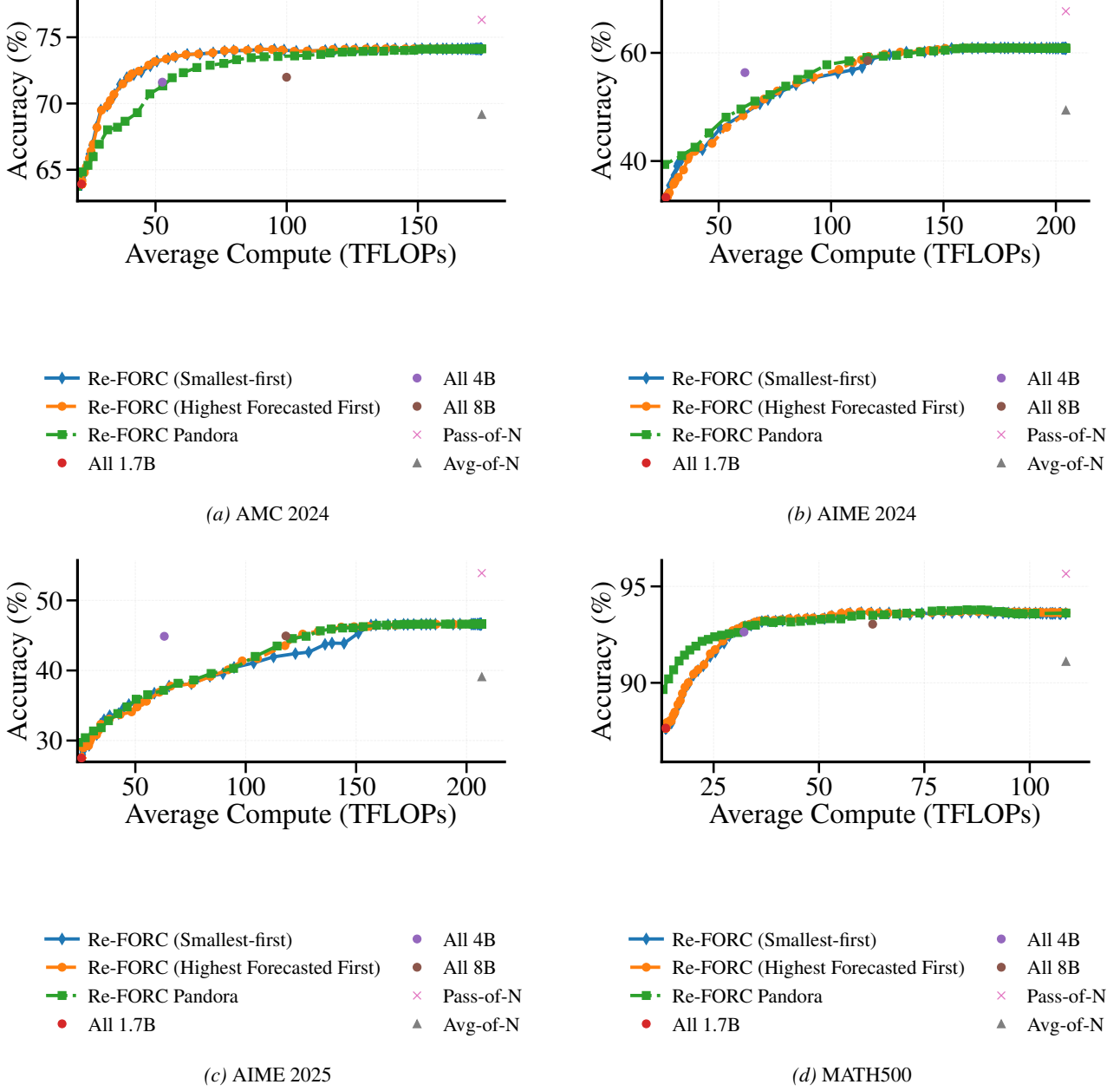


Figure 14. Per-dataset cost-aware model selection across the four math benchmarks.

#### B.4. Token-usage distribution per model size

Per-model-size, per-dataset breakdown of the token-distribution figure in Figure 5. Each subfigure shows the cumulative share of tokens spent versus cumulative problem difficulty for one Qwen3 model size, broken out across the four math benchmarks (AMC 2024, AIME 2024, MATH500, AIME 2025). Problems are ordered by difficulty from easiest to hardest based on solve rate. The dashed diagonal indicates uniform allocation; increasingly convex curves indicate selective compute use that concentrates effort on harder problems. On difficult datasets (like AIME 2024 or AIME 2025) the models preferentially allocates computation to easier problems for high  $\lambda=4.0 \times 10^{-4}$ . In contrast for an easier dataset (Math500), the curves for 8B model are increasingly convex for increasing  $\lambda$  (allocating a smaller fraction of compute to easier problems than to harder ones).

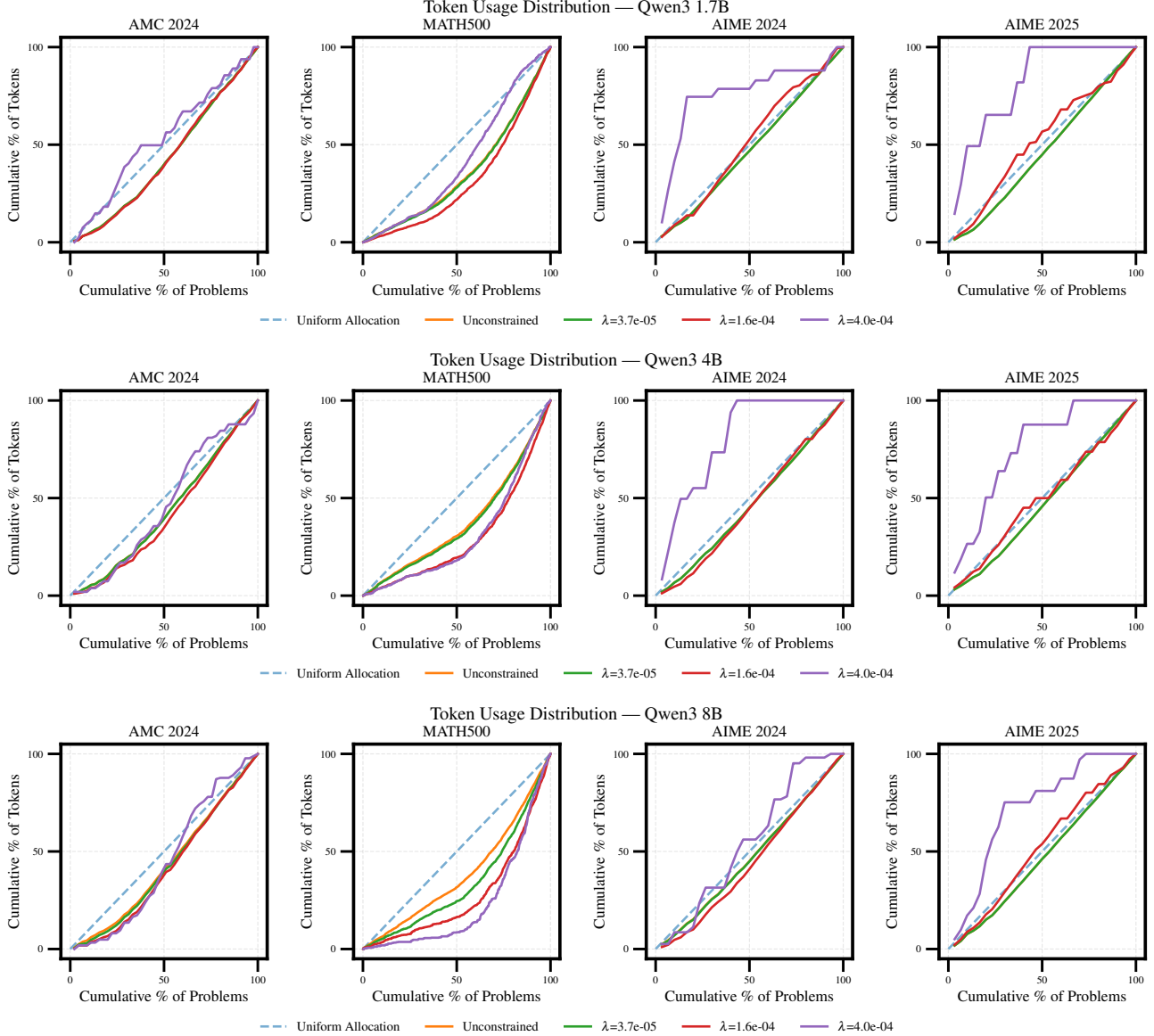


Figure 15. Token-usage distribution per dataset for the 1.7B Model (Top Row), 4B model (Middle Row) and 8B model (Bottom row).

### B.5. Test-time scaling per dataset

Per-dataset breakdown of the test-time-scaling experiment in Figure 4. Each row shows accuracy vs. average tokens used for Qwen3 1.7B, 4B, and 8B on a single benchmark. Baselines include Avg-of- $k$ , majority vote, best-forecasted-of- $k$ , three DeepConf variants (Online, Tail, Bottom-10%), and Pass@ $k$  as an oracle upper bound.

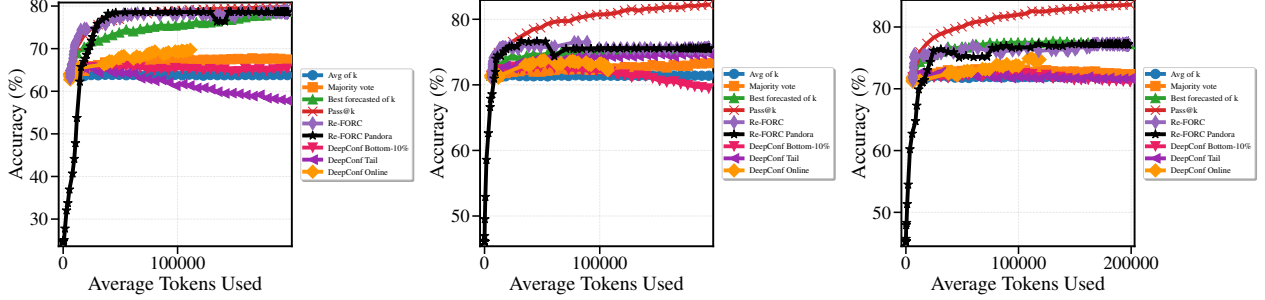


Figure 16. Test-time scaling on AMC 2024. Left: 1.7B, Middle: 4B, Right: 8B.

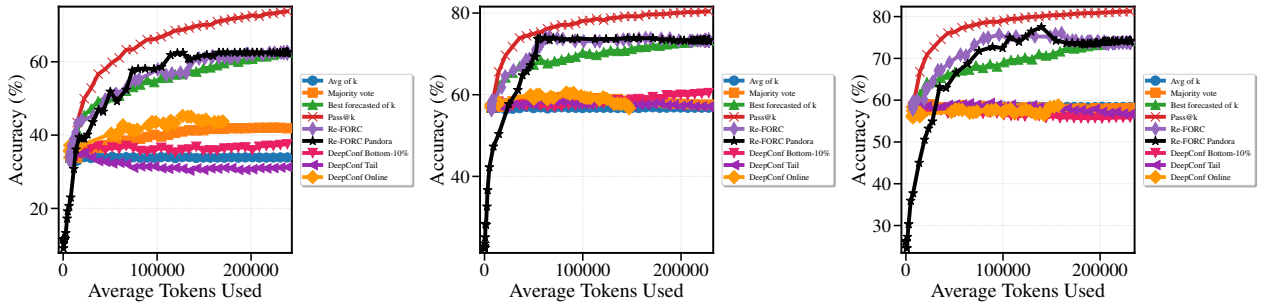


Figure 17. Test-time scaling on AIME 2024. Left: 1.7B, Middle: 4B, Right: 8B.

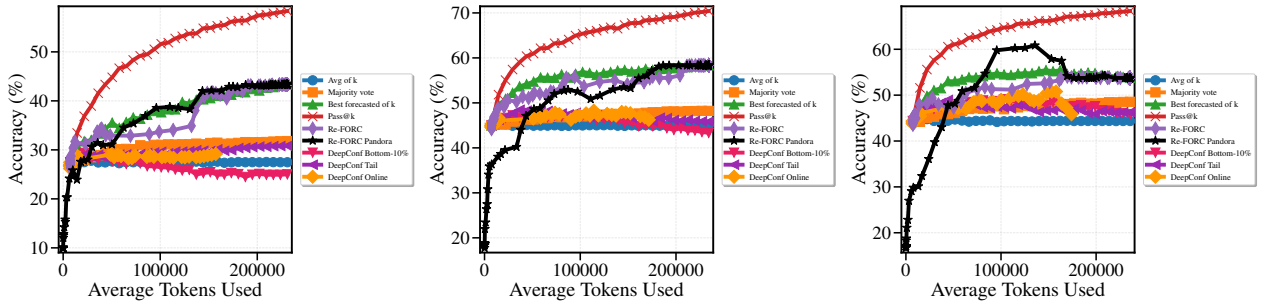


Figure 18. Test-time scaling on AIME 2025. Left: 1.7B, Middle: 4B, Right: 8B.

### B.6. DeepConf baseline implementation details

For the DeepConf baselines (Fu et al., 2026), per-token top-20 log-probabilities were recomputed post-hoc via a separate forward pass over the templated prompt (chat template and instruction) concatenated with the response, matching the generation-time conditioning context.